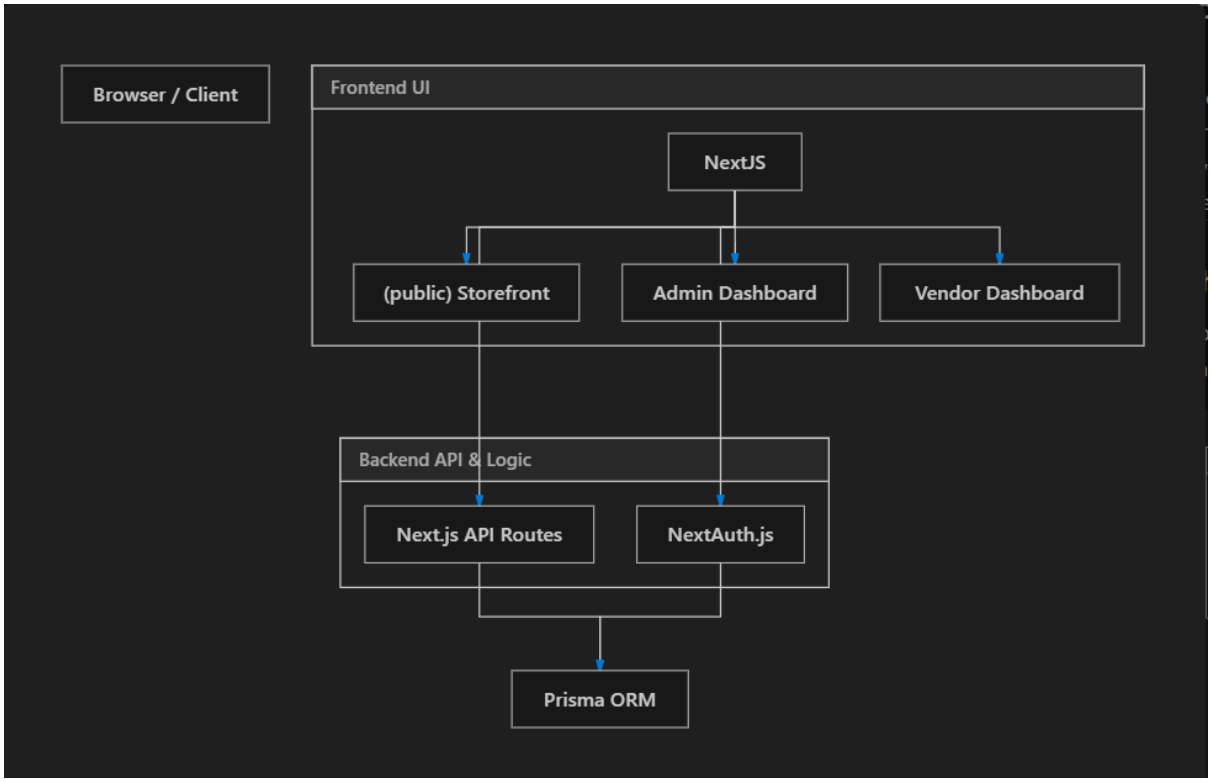
GoCart Project Synopsis

GoCart is a modern, full-stack, open-source multi-vendor e-commerce platform. It enables entrepreneurs to host a marketplace where independent vendors can register, list their products, and manage their sales, while customers can browse and purchase items from multiple vendors in a single checkout experience.

📐 System Architecture

The application follows a modern monolithic architecture utilizing Next.js App Router for both the frontend user interface and backend API routes.

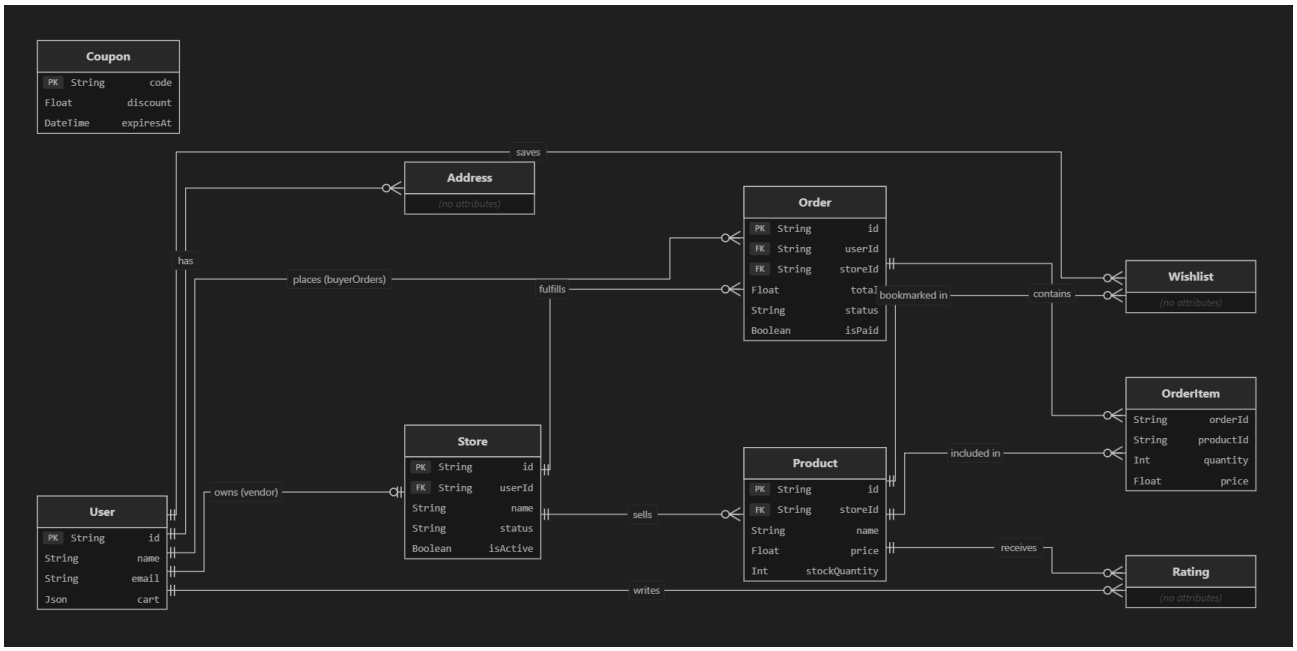


Architecture Highlights:

1. **Frontend**: Server-Side Rendered (SSR) and Client-Side Rendered (CSR) React components using Next.js 15. Styling is handled via Tailwind CSS. Global state (like the shopping cart) is managed by Redux Toolkit.
2. **Backend**: Next.js API routes (app/api/*) serve as a lightweight backend to handle data fetching, mutations, and integration with third-party services like Stripe.
3. **Database Layer**: Prisma ORM acts as the bridge between the Node.js backend and the PostgreSQL database, ensuring type safety and straightforward schema migrations.

🗄️ Database Schema (Entity Relationship)

The database is structured to support a multi-vendor environment, ensuring clear separation of concerns between buyers, vendors, and the platform administrator.



Core Entities:

- **User**: The foundational identity. A User can be a standard customer or they can upgrade their account by creating a Store, effectively making them a Vendor.
- **Store**: Represents a vendor's business profile. All products created by a vendor are linked to their Store ID, not just their User ID.
- **Product**: Items for sale, tightly coupled to a specific Store.
- **Order**: Uniquely, an Order in this system links a User (the buyer) and a Store (the seller). If a customer buys items from two different vendors at once, the system logic likely splits these into separate Order records per vendor to manage fulfillment properly.

🚀 Key Modules and Features

1. Storefront Module app/(public) & UI Components

- **Purpose**: The customer-facing interface.
- **Features**:

- **Landing Page**: Built using modular components like Hero.jsx, Banner.jsx, CategoriesMarquee.jsx, and BestSelling.jsx to create an engaging first impression.
- **Shopping Cart**: Managed globally via Redux Toolkit.
- **Checkout Flow**: Seamlessly integrated with Stripe. Includes OrderSummary.jsx and an OrderSuccessModal.jsx.
- **Modal-Driven Interactions**: Extensive use of modals (LoginModal.jsx, AddressModal.jsx, RatingModal.jsx) to keep users in context without forcing full page navigations.

2. Vendor Dashboard app/store

- **Purpose**: The dedicated portal for registered sellers.
- **Features**:
  - **Onboarding**: Users can apply to become vendors; their Store status defaults to "pending" until admin approval.
  - **Inventory Management**: CRUD operations for Products (add, edit, delete).
  - **Order Management**: View and update the status (PROCESSING, SHIPPED, DELIVERED) of orders placed for their specific products.
  - **Analytics**: Visualizing sales data using Recharts (e.g., OrdersAreaChart.jsx).

3. Admin Dashboard app/admin

- **Purpose**: The control center for the platform owner.
- **Features**:
  - **Vendor Approval**: Reviewing and approving/rejecting new Store applications.
  - **Platform Analytics**: High-level overview of total sales, active users, and system health.
  - **Coupon Management**: Creating and managing global or member-specific discount codes.

🔌 API Route Architecture (app/api/*)

The Next.js backend is cleanly separated into feature-specific domains:

- /api/auth: Handles NextAuth callbacks, sign-in, and sign-out logic.
- /api/products: Endpoints for fetching product catalogs, filtering, and vendor CRUD actions.

- /api/checkout: Generates secure Stripe Checkout Sessions for payment processing.
- /api/stripe: Crucially contains the **Webhook Handler** to receive asynchronous payment confirmations from Stripe and update the Order.isPaid status safely.
- /api/store: Vendor specific actions (applying for a store, updating store details).
- /api/coupon: Validation logic to check if a discount code is active, public/member-only, and meets the minimum purchase requirements.
- /api/wishlist & /api/user: Endpoints for managing user profiles, addresses, and saved items.

🧠 State Management (Redux Toolkit)

While Next.js handles server state, complex client-side interactions are managed via Redux Toolkit (lib/features/). The store is divided into several specific "slices":

1. **Cart Slice (cart/)**: The most critical slice. Handles adding/removing items, updating quantities, and calculating totals instantly on the client without waiting for database round-trips.
2. **Product Slice (product/)**: Likely caches product listings or handles complex filtering states on the storefront.
3. **Address Slice (address/)**: Manages the user's selected shipping address during the checkout flow.
4. **Rating Slice (rating/)**: Temporarily holds review data while a user is filling out the RatingModal.

🛠️ Technology Stack Summary

Layer	Technology	Purpose
Frontend Framework	Next.js 15 (App Router)	React framework for UI, SSR, and API routes.
Styling	Tailwind CSS v4	Utility-first styling for rapid, responsive design.
State Management	Redux Toolkit	Managing complex global state across multiple UI slices.
Database	PostgreSQL	Robust, scalable relational database.

Layer	Technology	Purpose
ORM	Prisma v6	Type-safe database interactions and schema management.
Authentication	NextAuth.js (v4)	Secure user login and session handling (credentials/OAuth).
Payments	Stripe	Securely processing customer payments & Webhooks.
Icons & Charts	Lucide React / Recharts	Clean UI elements and data visualization.
Testing	Puppeteer	End-to-End automated testing of user flows.